Design layer

the whole layer, as one document

Contents

1 HTTP Gun 4

1.1 Foundation 4
1.2 Pending updates 6
1.3 System at a glance 7
1.4 Local model 7
1.4.1 Values and refinements 8
1.4.2 Client and connection authorities 8
1.4.3 Body and capture authorities 10
1.5 Client configuration and lifecycle 12
1.5.1 Configuration validation and mode choice 12
1.5.2 Default settings 13
1.5.3 Client certificate identity 13
1.5.4 View composition and time authority 14
1.5.5 Supervision shutdown and mailbox lifetime 15
1.6 Admission and destination policy 15
1.6.1 Request admission order 15
1.6.2 Address classes and policy intersection 16
1.6.3 Resolution TLS and worker ownership 16
1.7 Connection pool and leases 17
1.7.1 FIFO lanes and bounded preparation 17
1.7.2 H1 readiness and H2 capacity 18
1.7.3 Lease release and retained handles 18
1.8 Body demand and response fidelity 18
1.8.1 Head admission and wire boundaries 18
1.8.2 Demand reads and terminal arbitration 19
1.8.3 Asynchronous application composition 19
1.9 Budgets and effect timing 19
1.10 Collection batching and failures 21
1.10.1 Collection and overflow 21
1.10.2 Bounded batch lifecycle 21
1.10.3 Failure meaning and serialization 21
1.11 Playback cassettes and redaction 22
1.11.1 Playback matching and cursor 22
1.11.2 Cassette schema and bounded file input 22
1.11.3 Secret ownership and whole-body rewriting 23
1.12 Recording and filesystem publication 23
1.12.1 Reservation order and acknowledged writer 23
1.12.2 Finish waiter and publication commit 23
1.12.3 Failure cleanup and operational limits 24
1.13 Observation and composition ports 24
1.13.1 Milestones and delivery selection 24
1.13.2 Caller and extension obligations 25
1.14 Foreign boundary and dependency upgrades 25
1.14.1 Native assumption inventory 25
1.15 Validation and retained capability scope 26
1.15.1 Evidence and tests by boundary 26
1.15.2 Validation profiles and gates 27
1.15.3 Public consumers and local fixtures 27
1.15.4 Comparison and benchmark method 28
1.15.5 Retained extensions and explicit exclusions 28

Glossary 28

HTTP Gun

1 HTTP Gun

1.1 Foundation

Generic HTTP exchanges with explicit ownership.

GOAL

One client serves ordinary and composed HTTP calls

HTTP Gun supplies Erlang/OTP applications with binary HTTP requests, process-owned response streams, finite collection, connection reuse, deterministic offline playback and explicit live recording. Ordinary applications and provider clients use the same public interface.

GOAL

Failure preserves decisions the caller must make

Request failures retain typed causes, conservative submission evidence and available response facts. The caller decides status semantics, application retry, framing and external effect reconciliation.

NO-GOAL

Application effects remain outside HTTP ownership

HTTP Gun owns no provider outcome, workflow continuation, agent checkpoint, transfer recovery, business idempotency, automatic request retry or ambient cookie/cache policy.

NO-GOAL

Gun and Cowlib own wire parsing

The package supplies no custom HTTP/TLS parser, dependency patch, WebSocket API, HTTP/3 API or server framework. It claims no bound on total VM memory or allocations that precede its admission checks.

INVARIANT

An admitted invocation performs one attempt

HTTP Gun never resubmits after invoking Gun's request API. Gun reconnection retry is disabled. Preparing a replacement unused connection before submission preserves the original attempt.

mechanism

INVARIANT

One process owns each body cursor

A body actor serializes every read, deadline and close over one cursor. Only the process that opened the response may read it. Every copied body capability targets that actor.

mechanism

INVARIANT

Every live connection has an admitted address set

Every new live connection uses a checked IP tuple and original TLS identity. Client and view policies intersect. No replacement connection bypasses address admission or silently falls back to Gun DNS.

mechanism

INVARIANT

Fixtures never trigger a live fallback

Script and cassette playback are offline. Missing, corrupt, incompatible, mismatched and exhausted inputs remain explicit failures.

mechanism

PRINCIPLE

Keep state and policy in Gleam

Gleam owns configuration, admission, pool state, body state, batch scheduling, matching, codecs, recorder coordination and filesystem policy. Foreign code binds Gun events and genuinely missing runtime/filesystem primitives. ADR-0001

PRINCIPLE

State limits at the boundary that enforces them

Message credit, post-parse header checks, retained-body limits and encoded fixture budgets describe different resources. A limit on admitted values must not be presented as a parser or whole-stack allocation guarantee.

PRINCIPLE

Expose independent completion authorities

HTTP completion, application consumption, local cancellation, observer delivery and recording publication remain separate facts. A failure in one does not invent success or failure in another.

1.2 Pending updates

build **Streamed upload producer contract** 2026-09-29

Retain bounded producer demand, finish/abort, early response, producer failure and request replayability. Buffered uploads are the current path. An arbitrary enumerable is insufficient evidence of backpressure.

9

build **Explicit redirect and decompression policies** 2026-09-29

Redirects require hop/deadline bounds, method/body transitions, origin/downgrade policy and credential stripping. Decompression requires encoded and decoded limits, incremental expansion bounds and truthful response headers.

9

build **Proxy route and credential identity** 2026-09-29

Retain explicit proxy routing and tunnel/proxy credentials in connection keys. Every supported path requires independent destination, TLS and lifecycle tests.

9

build **Optional cookie cache and SSE helpers** 2026-09-29

Cookie/cache adapters have caller-owned policy, storage, capacity and lifetime. A generic SSE helper is pure framing over HTTP bytes. Provider framing remains in its provider package until this helper exists.

9

ruling **Choose crash-durable fixture publication** 2026-10-06

Atomic visibility is implemented. A power-loss durability promise would require file and directory synchronization with explicit supported-platform semantics. It has no accepted implementation contract.

5

ruling **Resolve recording cleanup promise strength** 2026-10-06

Abort and failure attempt staging removal. The implementation ignores removal failures and can return while a blocked writer's cleaner waits. The public contract is either best-effort cleanup or typed cleanup completion; the owner has not ruled.

10

verify **Check carried-over defaults and native claims against code** 2026-10-17

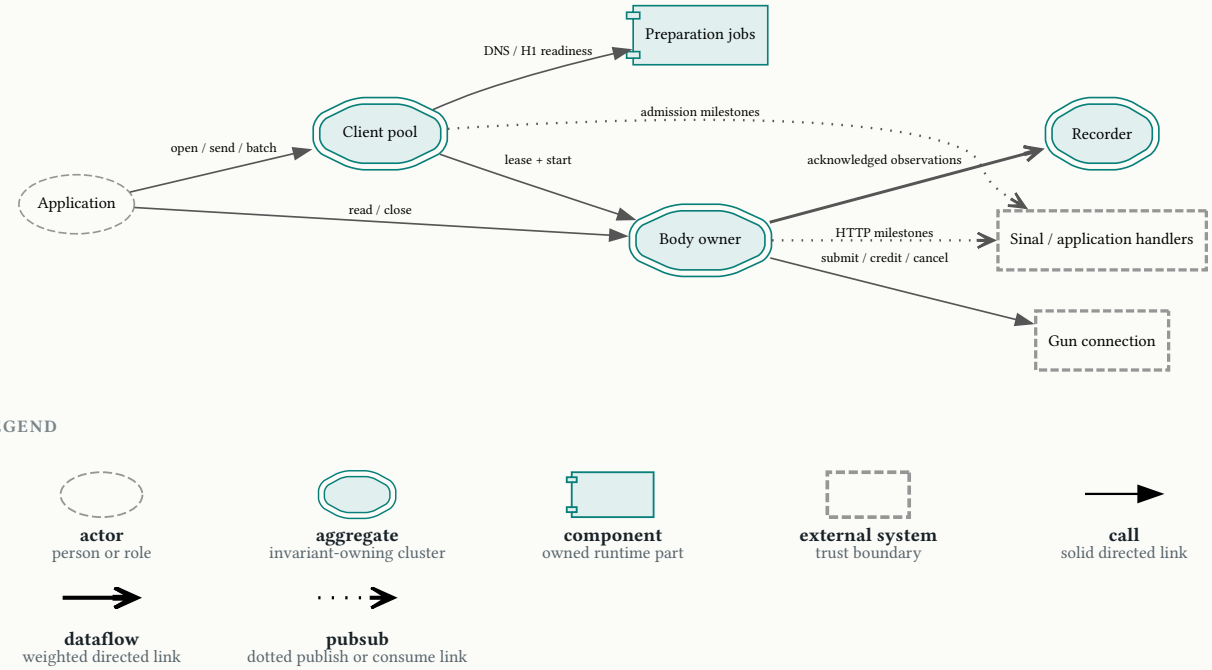
The default-settings table, the native assumption inventory and the destination and offline-playback rules were carried over from the retired usage, testing and dependency-upgrade guides. The compiled usage examples exercise the public paths. The stated default values, budgets and native assumptions have not been compared with the source.

1.3 System at a glance

One HTTP exchange context with separate invariant owners.

- The context contains client lifecycle, admission and destination checking, pool leasing, body ownership, collection and batching, playback and recording, observations and foreign boundaries. Their shared exchange vocabulary earns one context.
- The pool owns capacity and scheduling. A body actor owns response progress. A recorder owns capture ordering and publication. Those owners exchange typed messages without sharing mutable state.

VIEWPOINT RUNTIME · ALTITUDE L3 · COMPONENTS Runtime ownership and calls



Request data flows to its body owner; response messages never use the HTTP caller as Gun's reply target.

Unit	Authority and public boundary	Children
Client	`config`, `start`, `supervised`, `named`, views, `stop`	Configuration; view composition; client lifecycle
Admission and pool	Request validity, destination decisions and lease membership	Origin queue; resolution; H1 readiness; H2 capacity; release
Body	`open`, `with_response`, `body.next`, `next_within`, `close`	Head admission; demand; byte retention; terminal delivery; cleanup
Collection	`send`, `body.collect`, `batch`	Per-response overflow; shared batch budget; bounded workers
Fixtures	`testing`, `cassette`, `redaction`	Match keys; schema; capture reservations; writer; finalization; publication
Observation	`telemetry`, client labels and correlation	Milestones; delivery mode; application forwarders
Foreign boundary	Gun/OTP and filesystem operations	Event decoding; typed causes; atomics; time; scope cleanup; upgrade audit

- Deployment is one Erlang/OTP application with client-local actors and native Gun connections. There is no service endpoint, external database or deployment topology to configure. A cassette is an optional filesystem artifact rather than shared runtime storage.
- Applications translate HTTP results into LLM, MCP, OIDC or business vocabularies. HTTP Gun imports none of those packages. Sinal supplies correlation and observation delivery; file_streams and simplifile supply filesystem mechanisms.

1.4 Local model

Values, authorities and state links within the HTTP exchange context.

1.4.1 Values and refinements

Value	Meaning and refinement	Ownership
Request	`Request(BitArray)` preserves method, scheme, host, port, target, ordered headers and whole body bytes. Construction is pure.	Caller authors; admission validates
Response	Final status and ordered duplicate headers plus a body capability or collected bytes. Trailers remain separate. Non-success statuses remain data.	Body owner observes
Origin	Lowercase unbracketed host, effective 80/443 or explicit port, TLS choice. Port is 1–65535.	Pool derives at validation
Config and view	Opaque immutable configuration and per-handle overrides. A view contains optional timeout, deadline, idle bound, cancellation, body limit, policies and correlation.	Caller authors; client startup validates config
Address	IPv4/IPv6 integer components. Invalid components classify Reserved. Policy uses Public, Loopback, Private or Reserved.	Resolver observes; destination policy decides
Failure	Opaque reason, submission evidence, optional final status and retained response headers. Kind is closed; detailed reason/cause unions may grow.	Boundary that detects failure
Deadline and token	Opaque VM-local monotonic instant; shared latched cancellation capability. Neither is durable or part of cassette matching.	Caller owns scope
Script and cassette	Ordered request/reply values with bytes, heads, trailers or typed terminal failure. A script's finite estimated data budget differs from file bytes.	Caller or file decoder supplies
Request id and correlation	Fresh UUID v4 invocation identity and independently supplied caller correlation. Neither grants retry permission.	HTTP Gun derives id; caller supplies correlation

- Authority is held by process destinations and opaque capabilities. Names identify a supervised client's current incarnation. They are not credentials or durable exchange identifiers.
- Commands include startup, open, read, close, stop, reserve capture, write observation, finish and abort. HTTP heads/bytes/trailers, connection capacity/down signals and local timers are observed events. Observation milestones are best-effort notifications rather than an event-sourced history.
- The model census covers four stateful authorities and nine value families. There is no application business entity or remote execution ledger. Local request references index pending work and do not survive restart.

1.4.2 Client and connection authorities

Client phase running draining closed

Client pool aggregate stateful HTTP exchange owned by HTTP exchange

The aggregate owning one client's capacity, admission and lifetime.

ATTRIBUTES

derived **Phase** · Client phase client-phase · client-lifetime
Established by startup and stop transitions; derived at mutation.

authored **Configuration** · Validated client policy
Immutable startup limits, trust, protocol, destination, redaction and observation choices.

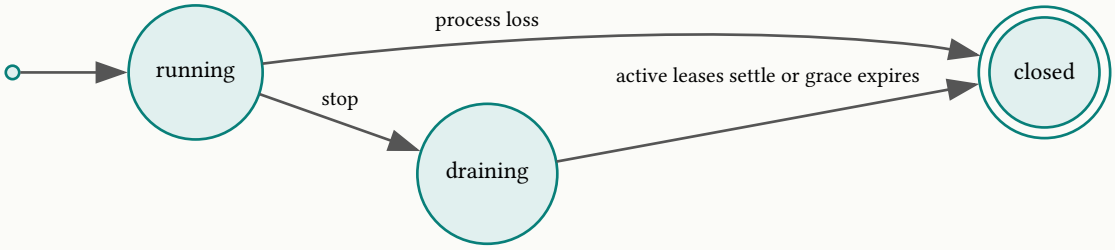
derived **Capacity membership** · Connections, owners and origin queue
Current owned resources and pending work; updated together at admission and release.

RELATIONSHIPS

`1 : 0..n` owns connection reservations

`1 : 0..n` tracks body owners until close or process death

STATE MACHINE Client draining



Closed includes an absent process. A named handle reaches a replacement pool without reviving old exchanges.

Connection phase resolving · connecting · ready · checking · checked · removed

Connection reservation

entity · stateful · HTTP exchange · owned by HTTP exchange

A pool-owned identity for preparation and a native connection's capacity.

ATTRIBUTES

derived **Phase** · Connection phase connection-phase · connection-lifetime
A transition-derived scheduling state; removed means absent from the pool.

derived **Origin** · Canonical origin
Derived from the first request without changing original HTTP authority.

observed **Checked answer** · Complete admitted address set
The resolver's answer checked before opening; retained for every reuse-policy intersection.

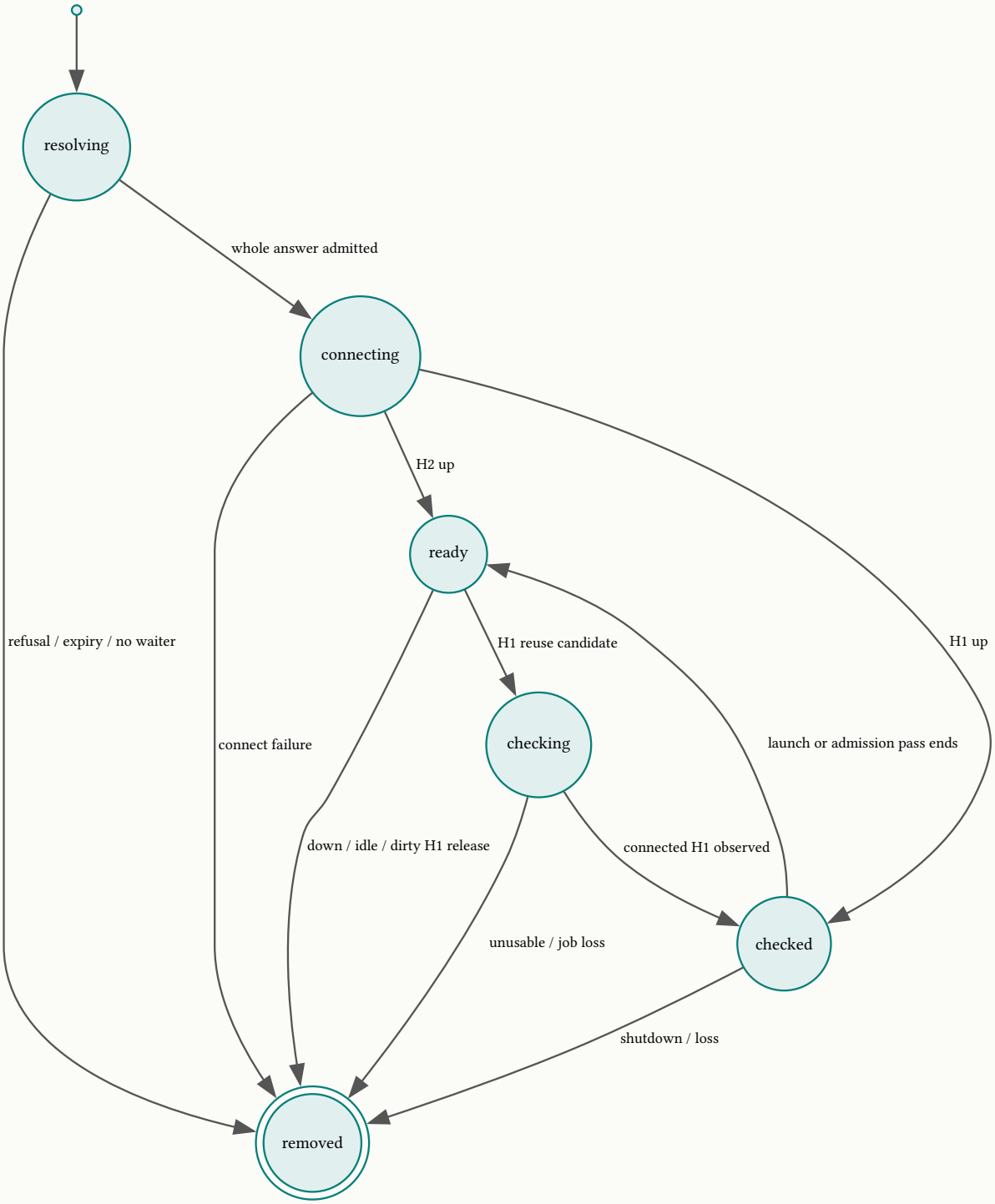
derived **Used capacity** · Active leases
One for H1; concurrent stream count bounded by local and peer capacity for H2.

RELATIONSHIPS

`n : 1` belongs to one client pool

`1 : 0..n` carries stream leases

STATE MACHINE Connection preparation and reuse



Checking and Checked apply to H1. A checked observation is consumed by one admission pass.

- A vacant client/connection is represented by process absence or removed membership. Restart creates new authority. Checked readiness can be revoked by job failure or discarded at the end of an admission pass.
- H2 Ready includes protocol and capacity. Initial capacity is zero until peer SETTINGS. Existing used streams may exceed a newly reduced peer limit; the pool admits no additional stream until usage falls below it.

1.4.3 Body and capture authorities

Body phase opening reading finished rejected closed

Body owner aggregate stateful HTTP exchange owned by HTTP exchange

The aggregate serializing one request attempt and response cursor.

ATTRIBUTES

derived **Phase** · Body phase body-phase · body-lifetime
Finished retains success or failure; Rejected retains an opening failure. Closed means actor absence.

observed **Opening process** · Read authority
The caller identity fixed at admission; monitored for its complete lifetime.

observed **Head and terminal outcome** · Observed HTTP facts
Admitted final status and head plus eventual trailers or typed failure.

derived **Demand and retained bytes** · One read wait and bounded byte queue
Current cursor, queue bytes and flow credit; updated by reads and delivered events.

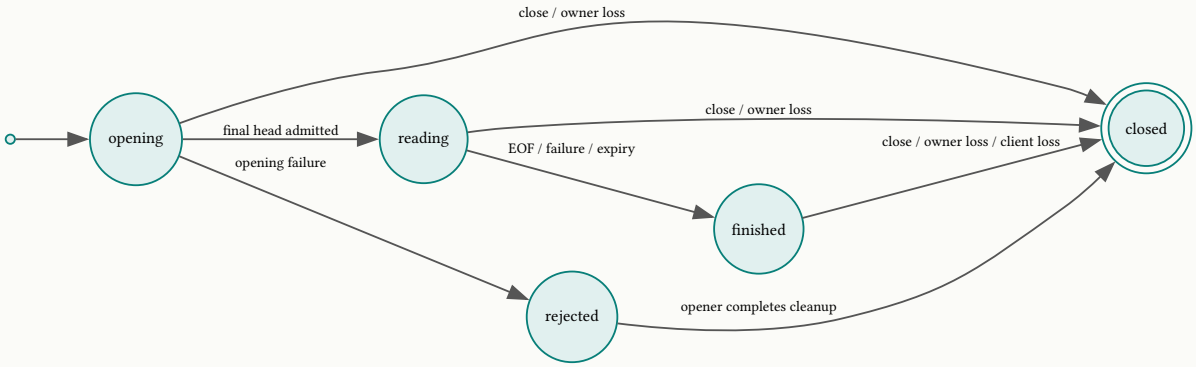
RELATIONSHIPS

`n : 1` is tracked by one client pool

`1 : 0..1` holds one stream lease

`1 : 0..1` feeds one capture reservation

STATE MACHINE Body lifetime and terminal retention



Reading failures settle in Finished with an error. Closing settles unfinished HTTP and stops the actor; copying a capability never transfers ownership.

Recording phase active closing sealing sealed broken

Recording session aggregate stateful HTTP exchange owned by HTTP exchange

The aggregate owning exchange order, writer acknowledgment and publication.

ATTRIBUTES

derived **Phase** · Recording phase recording-phase · recording-lifetime
Finalization transitions derive from reservations, jobs and publication acknowledgment.

derived **Exchange positions** · Ordered capture reservations
Monotonic zero-based positions assigned at reservation, not at HTTP completion.

derived **Storage budget** · Encoded bytes plus held bodies
Conservatively charged before enqueueing fragments or retaining whole-body redaction input.

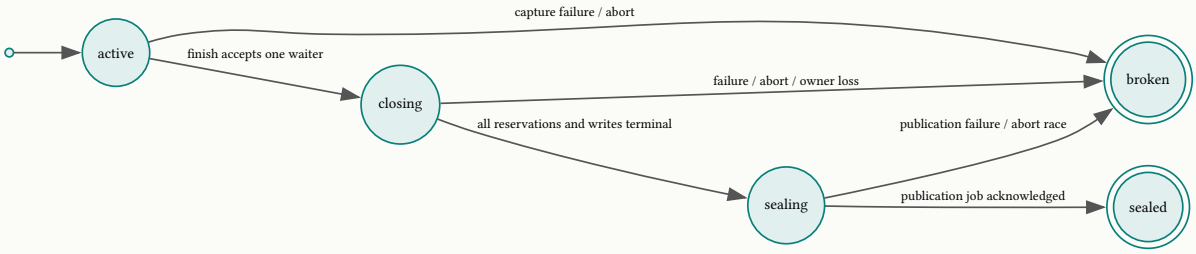
authored **Publication choice** · Destination and replacement policy
Explicitly chosen at record startup; destination is not part of safe error detail.

RELATIONSHIPS

`1 : 0..n` owns capture reservations

`1 : 0..1` owns a current writer and a finish waiter

STATE MACHINE Recording finalization



A waiting caller may leave Closing or Sealing without aborting capture. Atomic publication remains the irreversible commit point.

- Capture stream state is AwaitHead, Receiving, Buffering or Ended. Receiving retains chunk framing; Buffering retains the whole body for redaction. Head starts Receiving or Buffering; bytes advance that state; completion/failure ends it. Invalid observation order breaks the recording.
- Read wait presence, capture acknowledgment presence and HTTP phase are independent state axes. At most one read wait and one outstanding capture acknowledgment exist per body. A terminal HTTP result cannot become unsuccessful solely because its consumption occurs after the deadline.

1.5 Client configuration and lifecycle

Pure setup, shared views and explicit resource lifetime.

Client configuration boundary

contract

Validate complete settings before resources start.

responsibility

Own defaults, pure setters and startup validation.

interface

`config.default`, `with_*`, `validate`; `start` or `supervised(config, name)`.

interactions

The validated settings feed pool, body owners, destination preparation and recorder startup.

invariants

A setter starts no process. Invalid settings cannot produce a live pool.

failure

InvalidConfig retains ConfigError; startup failure is StartFailed.

1.5.1 Configuration validation and mode choice

- `Config`, `Policy`, `Redaction` and `RecordOptions` are opaque caller-built values. Value-first setters permit extension without forcing callers to construct all fields. Returned Buffered and Stats records are read by label; consumers leave room for added fields. ADR-0002
- Positive capacities are required for connections, per-origin connections, H2 streams, open bodies, header bytes/count, buffered bytes and batch bytes. Queue, request-body and response-body limits may be zero. Connect, pool and pooled-idle timeouts are positive; shutdown may be zero. Request/idle timeout may be explicit Infinity.

- In-memory trust anchors must be nonempty whole-byte values and their list must be nonempty. Invalid host-list entries fail configuration validation. Actual certificate or CA-file validity is established by OTP when connecting, rather than by pure settings construction.
- Live startup, offline startup and recording startup return the same Client type. Recording also returns a Recording capability. Application startup chooses a mode explicitly; requests read no environment variables and contain no automatic mode fallback.

BEHAVIOR

Invalid startup has no usable client

Given the complete client settings are invalid

When the caller starts that client

Then startup returns the typed configuration failure

Then no usable client capability is returned

level: boundary area: Client configuration

1.5.2 Default settings

- Every timeout, deadline and wait is a `gleam/time/duration.Duration` held as whole milliseconds; a sub-millisecond remainder rounds away from zero. Unbounded request or idle timeout is `config.Infinity`; a bounded one is `config.After(duration)`. Lifting the request timeout never lifts the connect, pool or idle timeout, and each request phase is also capped by the time left in the request.
- `config.validate` checks settings without starting a process. `start` returns InvalidConfig for an out-of-range value. Capacity and timeout defaults appear in Budgets and effect timing.

Setting	Default	Selected with
Destinations	Public addresses only	`config.allow_loopback`, `config.with_destination`
Plaintext `http://`	AllowPlaintext	`destination.with_plaintext`
Destination per view	Not required	`config.require_view_destination`
TLS client identity	None	`config.with_client_identity`
Protocol and server trust	HTTP/1.1; system CAs; peer and host name verified	`config.with_protocol`, `config.with_trust`
Cassette redaction	Credential headers `authorization`, `proxy-authorization`, `cookie`, `set-cookie`, `x-api-key`, `api-key`, `x-goog-api-key`	`config.with_redaction`
Observations	`sinal.emit`, following the application's routes	`config.with_observations`, `config.without_observations`
Client label in events	None	`config.with_label`

1.5.3 Client certificate identity

- `client_identity.from_pem` admits a nonempty leaf-first PEM certificate chain and exactly one unencrypted PEM private key into an opaque immutable Client identity snapshot. It performs no filesystem or network operation. Malformed certificate/key input and encrypted private keys produce typed errors without retaining the input in diagnostics. OTP owns PEM/DER parsing and TLS algorithm support. ADR-0011
- `config.with_client_identity` selects that snapshot for the client's TLS connections. Identity omission preserves ordinary server-authenticated TLS. Server trust and HTTPS hostname verification remain independent of client authentication. Plaintext connections never transmit certificate/key material.
- Each client has its own pool and one immutable identity. The client pool plus canonical origin partitions authenticated connections. Views share that identity and cannot replace it. Every reconnect uses the same admitted bytes; replacing an application file cannot change an existing client's identity.
- Rotation starts a new client with a newly admitted identity, directs new application work to that client, and stops the old client under its existing drain contract. In-flight old-client work uses its old authenticated connection until completion or drain expiry. Applications that need old results await and consume them before stop; shutdown grace covers active HTTP leases, and pool exit ends retained body access. Applications own overlap capacity and credential acquisition; HTTP Gun owns no file watcher or credential registry.

- PEM admission establishes decoding, not certificate trust, expiry, permitted usage or certificate/key pairing. TLS establishes those connection properties. TLS refusals retain existing typed transport causes and submission evidence. TLS 1.3 can reject authentication after the client considers its handshake complete, so a refusal can be MaybeSent after the Gun request call. No request is replayed.
- Identity material remains only in caller-owned input/configuration and the native TLS state that needs it. HTTP Gun error descriptions, observations and cassettes contain no configured certificate or private key. Opaque values are not encrypted memory; callers must not print arbitrary configuration/runtime state.

BEHAVIOR

TLS presents only the configured client identity

Given a server requires a trusted client certificate

When a configured client performs a secure request

Then the server observes that client's admitted certificate and returns its ordinary response

Then missing or rejected identity produces a typed failure without automatic retry

level: boundary area: Client identity

BEHAVIOR

Rotation preserves in-flight identity

Given an old client has an active exchange and a new client has a different identity

When application work switches to the new client and the old client drains

Then new-client requests authenticate with the new identity on separate connections

Then the active old-client exchange retains its old identity until completion or drain expiry

Then reconnection by an unstopped client retains its original identity snapshot

level: boundary area: Client identity

1.5.4 View composition and time authority

- A Client view copies only a handle and its view values. Scalar setters replace their prior value. Destination setters append policies whose intersection is the Destination policy applied at every operation. Later correlation replaces earlier correlation.
- with_cancellation replaces the view's cancellation tokens with one token. with_additional_cancellation retains existing tokens and adds a distinct token. Any retained token can cancel requests through that view without cancelling another token or changing another view.
- `cancellation.with_token` creates one Cancellation token process for its lexical scope. Cancellation kills the token process, which makes every copy remain cancelled. Current pending and body owners receive their monitors' death signal; a later attempt sees the dead token before submission. Pending rejection, admission handoff and body termination remove every owned token monitor. Adding cancellation creates no process and takes no token lifetime ownership. ADR-0012 records the library-composition requirement. Callback return, exception or creator death ends the token. Applications bound the number and lifetime of token scopes.
- A view's relative request timeout or absolute deadline replaces the startup request timeout, whether shorter or longer. With both supplied, the earlier applies. With neither, the startup timeout starts from the invocation's monotonic entry instant before admission.
- The absolute Deadline is VM-local and cannot be serialized as durable policy. Reading remaining duration never renews it. Every public duration is converted to whole milliseconds with sub-millisecond remainders rounded away from zero. Nonpositive Deadline creation is immediately expired.
- Views do not change immutable client identity, connection trust, protocol, resolver or startup socket timeout options. Idle read can be replaced per view; pooled socket send settings continue to use startup idle timeout. The body owner's independent overall timer still cancels an earlier request budget.
- `with_body_limit` replaces collection policy only. Negative view collection limits fail before submission. A zero limit permits an empty body; Truncate can return an empty prefix on overflow.

BEHAVIOR

Independent cancellation authorities remain effective

Given a client view retains application cancellation and additional operation cancellation

When either cancellation is requested

Then unfinished requests through that view terminate with cancellation and their existing submission evidence

Then a later attempt through that view fails before submission

Then the other cancellation authority and other client views remain unchanged

Then the request removes all cancellation monitors when its ownership ends

level: boundary area: Client views

1.5.5 Supervision shutdown and mailbox lifetime

- `start` links the pool to its caller. `supervised(config, name)` supplies a child specification; `named(name)` is pure and targets the current registered process. During restart absence, operations return ClientClosed with NotSent. An old body is never revived by pool restart.
- `stop` refuses new admission, fails queued work with ClientClosed/NotSent and waits for active leases within the Shutdown grace. Grace expiry locally closes unfinished bodies and native connections. Completed retained body actors lose the client authority when the pool exits. Repeated stop is harmless.
- Shutdown grace bounds the planned drain. Serial body close calls, actor scheduling and synchronous observation handlers are additional work; this is not a hard real-time stop bound.
- Pool notifications go to the pool actor. Gun replies go directly to the body owner. Public calls monitor only their own target, consume the typed result and flush that monitor on completion. Response data and stale monitor notifications are not forwarded to the HTTP caller.

BEHAVIOR

Stop ends admission before draining

Given the client has pending and active work

When the caller stops it

Then new and waiting attempts fail before submission

Then active exchanges may finish within the drain period

Then remaining exchanges are locally closed when draining ends

level: boundary area: Client lifecycle

1.6 Admission and destination policy

Refuse invalid or unauthorized attempts before submission.

1.6.1 Request admission order

- The invocation entry instant is captured before caller-side validation. Already cancelled tokens, expired absolute deadlines and negative body limits fail early. Body bit size must be divisible by eight. The method and lowercase header names must be tokens. Origin/port and target reject invalid forms.
- Paths start with a slash. Host, path and query reject whitespace/control forms checked by admission. Header values reject CR/LF/NUL and delivered request headers also pass the shared token/control/size check. Limits count name plus value bytes rather than wire delimiters.
- The pool checks required view destination and request body/header limits, then reserves capture when appropriate. Deadline, monitors and timers belong to the pending entry before leasing. A new request never overtakes an existing request in its origin lane.
- The Origin is cached after validation. Pending entries, owner indexes, cancellation indexes, FIFO links and waiting count mutate together. Removing expired/dead work unlinks it immediately, including work behind a blocked origin head.
- Connecting reservations are distinct from waiting queue entries. A request starting a connection may wait for its connect timer beyond ordinary checkout expiry, while its overall deadline still caps the attempt. Resolving and connecting reservations consume connection capacity.

BEHAVIOR

A full queue refuses before submission

Given no eligible lease is available and waiting admission is full

When another request seeks admission

Then the request fails with capacity refusal and NotSent

Then existing waiting requests preserve admission order

level: boundary area: Request admission

1.6.2 Address classes and policy intersection

- Public-only is the default. Loopback and private networks require explicit opt-in. Reserved addresses remain refused under every option. Host lists restrict the original case-insensitive host and optional port; they do not authorize TLS downgrade, DNS aliases, paths or application access.

Class	Address classification	Admission
Loopback	127/8 and IPv6 loopback	Explicit client/view permission
Private	RFC1918, carrier-grade 100.64/10 and IPv6 ULA	Explicit private permission
Reserved	Invalid components; unspecified, link-local, multicast/broadcast; documentation/benchmark ranges; 192.0.0/24; 192.88.99/24; 6to4, Teredo, ORCHID, 100::/64, 3fff::/20; remaining special IPv6 outside public global unicast	Always refused
Metadata exceptions	100.100.100.200 and fd00:ec2::254	Reserved even with private permission
Embedded IPv4	IPv4-mapped IPv6 and 64:ff9b::/96	Inherit embedded IPv4 class
Public	Other admitted IPv4 and remaining 2000::/3 IPv6 global unicast	Default permitted

- A host list entry is host, host:port, bracketed IPv6 or bracketed IPv6:port. An empty list admits no host. Whitespace, URI delimiters, empty names and invalid ports fail validation. `destination.check` checks literal addresses and the host list without resolving hostname answers.
- Plaintext is AllowPlaintext, PlaintextToLoopbackOnly or RequireTls. A loopback-only plaintext choice checks every address in the complete answer, including reuse. HTTPS remains subject to address policy and verified TLS.
- A client shared by tenants with different destinations must admit the union of their destinations. A call that holds the client without a tenant view reaches that union, so `require_view_destination` makes the client policy only bound what a view may narrow to. A view that admits more than the client receives only what both admit.
- Every startup policy and appended view policy must admit the original host/port and every resolved address. No view can widen the client. For `require_view_destination`, one policy must have a host list or refuse an address class admitted by the client. A scheme-only restriction never satisfies it. The required-view check also runs offline. ADR-0004

BEHAVIOR

A view must select its own destination

Given the client requires a view destination

Given the view only tightens the plaintext rule

When the caller opens a request through that view

Then the request is refused with ViewDestinationRequired and NotSent

Then no address resolution or request submission begins

level: boundary area: Destination policy

1.6.3 Resolution TLS and worker ownership

- Each new hostname connection obtains A and AAAA answers once, sequentially within one connect budget. A family with no records contributes an empty list. A lookup failure fails closed even when the other family succeeded. A fully empty answer fails.

- The Checked answer includes every returned address. The first admitted address is the selected IP tuple. There is no alternate-address retry, Gun DNS fallback or HTTP Gun cross-connection DNS cache. OTP may apply its own resolver cache.
- Original host authority and TLS server identity remain distinct from the selected IP tuple. TLS uses peer verification and HTTPS hostname matching. IP literals omit the SNI option entirely; they never use SNI disable. System trust, PEM CA file and DER anchor list replace one another. A caller Host header changes no resolution/TLS authorization.
- Each resolving reservation has one Gleam guardian and one linked worker bounded by connection capacity. Pool death or job deadline kills the worker. Request cancellation/owner death removes unused reservations; late answers cannot reopen removed membership. Other origin lanes can progress while a callback blocks.
- Custom resolvers are trusted callbacks supplied complete host/budget input. They own resources they spawn and their memory allocations. Killing the worker does not claim cleanup of independently spawned callback resources.

BEHAVIOR

Mixed address answers fail closed

Given one resolved address violates an applicable destination policy

When the complete answer is considered for a new connection

Then the entire attempt is refused before submission

Then an allowed address in that answer is not used as a fallback

level: boundary area: Address admission

1.7 Connection pool and leases

Reuse eligible capacity with explicit preparation and release.

Pool lease boundary

contract

Own finite transport and retained-body admission.

responsibility

Serialize queue membership, preparation reservations, connection usage and owner membership.

interface

A validated invocation receives one body owner or a NotSent refusal. Stats expose connections, open bodies and queued requests.

interactions

DNS/H1 jobs report to this incarnation; body owners release leases and report closure separately.

invariants

One H1 lease per connection; H2 usage respects observed and local capacity; retained handles consume finite admission.

failure

Connection loss removes membership; admitted bodies receive their own conservative failure.

1.7.1 FIFO lanes and bounded preparation

- Live/recording scheduling uses one FIFO Origin lane per canonical origin. An admission pass stops at a blocked lane head and considers other lanes. Lanes that make progress rotate their position. Playback uses one global session lane to preserve matching order.
- Reuse comes before opening another connection. If global capacity is full, an unused ready connection for another origin may be evicted when that origin has no waiter. Idle sockets are a cache rather than a permanent capacity claim. ADR-0003
- Resolving/connecting/checking slots are finite. Only one new preparation exists per origin while that origin already prepares a connection. At most one H1 readiness job exists per connection. A readiness job also reserves an open body slot to preserve admission order at capacity one.
- Work crossing process boundaries captures only required values. A batch worker captures one input and its operation. Release/capture callbacks capture message destinations. They do not capture pool pending queues or completed batch results.

1.7.2 H1 readiness and H2 capacity

- An H1 lease is exclusive; the package does not pipeline independent requests. Before idle reuse, a bounded Gleam job calls supported `gun:info` and accepts connected HTTP state. Checking never blocks the pool. Checked is valid for one immediate admission pass and is reset if a body cannot launch.
- An unusable idle H1 connection is discarded before Gun request invocation. The original request retains position, deadline and cancellation while replacement preparation proceeds. A peer may still close after inspection; that submitted failure remains MaybeSent without retry.
- Http1 selects H1. PreferHttp2 negotiates H2 over TLS with H1 fallback; plaintext uses H1. RequireHttp2 refuses H1 negotiation and supports plaintext H2 prior knowledge. HTTP/2 receives no application stream before initial peer SETTINGS. Capacity is the lower of peer and configured stream limits.
- Gun owns H2 flow control, framing, GOAWAY and stream state. The pool uses supported up/settings/down events. Eligibility inspection is not atomic with GOAWAY or request submission. A race returns conservative failure and performs no hidden replay. ADR-0008
- An unexpected body-owner initialization failure may lack a stream reference; the pool closes that connection and affected siblings can fail. Healthy H2 sibling preservation applies to ordinary stream cancellation, not to lost connection ownership.

1.7.3 Lease release and retained handles

- HTTP termination releases the Stream lease exactly once. The Open body slot remains occupied for a finished explicit handle until close or process death. Stats therefore distinguish finished handles from used transport streams.
- Clean H1 completion allows reuse unless request or response contains a case-insensitive comma-separated Connection close token. Early H1 cancel/failure retires the exclusive connection. H2 cancellation cancels that stream and preserves healthy siblings.
- Retained bytes/trailers and terminal failure are readable after HTTP completion, even if the deadline subsequently passes. Closing clears cursor authority and is idempotent. Owner death and client loss converge on the same local cleanup.
- A ready unused connection gets one generation-marked idle timer. Reuse invalidates its old idle reference. Stale timers cannot close an in-use connection.

BEHAVIOR

Closing one H2 body preserves siblings

Given healthy independent exchanges share an H2 connection

When one owner closes its unfinished body

Then that exchange settles as local cancellation

Then healthy sibling exchanges remain usable

level: boundary area: Connection leases

1.8 Body demand and response fidelity

One cursor consumes generic bytes without a forwarding reader.

1.8.1 Head admission and wire boundaries

- `open` returns only after final response headers pass admission. Informational heads are checked then discarded. Final status, duplicate ordered headers, bytes and trailers are preserved. Gun applies HTTP empty-body/framing rules. Protocol upgrades fail as unsupported request outcomes.
- The shared delivered-header check counts every pair and every name/value byte, including the final pair. Names are tokens; values reject controls except HTAB. Informational, final and trailer lists each pass it independently. There is no cumulative informational-response count limit beyond the request/idle budgets.
- Gun receives the configured ordinary-header count for H1 and one extra H2 allowance for the status pseudo-header. HeaderLimitReached represents a dependency rejection without a measured value. LimitExceeded is reserved for measurements HTTP Gun actually made.
- Admission occurs after Gun/Cowlib parsing. It does not bound raw status lines, whitespace/delimiters, TLS buffers, HPACK expansion or parser allocations. Gun discards reason phrases, including experimentally accepted control/mixed-terminator forms. HTTP Gun cannot reject discarded data through this event API. ADR-0008
- Invalid status/framing can surface as dependency crash or PeerClosed without precise raw-wire diagnosis. Close-delimited TLS EOF without close_notify can be indistinguishable from truncation. Consumers needing proof of completeness require declared HTTP framing and payload validation.

1.8.2 Demand reads and terminal arbitration

- The opener is the sole read owner. If a **Read wait** already exists, another read returns ReadConflict before owner validation. Otherwise another process returns WrongOwner. Another capability holder may close the shared body.
- `next` waits for Chunk, End(trailers) or Failure. `next_within` adds a local wait; expiry returns None and removes only that read wait. Already admitted bytes and outstanding credit remain. A polling read still grants demand before local expiry is honored.
- The owner maintains one outstanding **Flow credit**. A reader drains admitted queued bytes before requesting more data. There is no actor that eagerly reads and forwards chunks into another unbounded mailbox.
- Every delivered data event decrements credit and undergoes buffered-byte admission before retention. A single chunk must fit. Message credit and supported H2 receive windows do not establish a byte-allocation bound inside Gun/Cowlib/OTP.
- Overall expiry, idle expiry, cancellation, close, owner/client death and native failure settle through the owner. Queued accepted bytes can be delivered before the terminal failure; a failed stream does not fabricate successful trailers. HTTP completion cancels its deadline timer unless capture is still pending.
- `with_response(client, req, on_failure, run)` maps opening failure to the caller's error type. Its callback returns Result and scope cleanup closes on success, failure or exception. Callback code owns read/application failure mapping; arbitrary callback computation is not interrupted by the HTTP timer after EOF.

BEHAVIOR

A local read wait preserves the stream

Given an unfinished response has no available bytes

When the local read wait expires

Then that read returns no event

Then later reads continue on the same cursor

level: boundary area: Response demand

BEHAVIOR

Completed HTTP survives later deadline passage

Given a response has already completed with retained bytes and trailers

When the owner reads after its request deadline

Then the completed response remains readable

Then deadline passage does not replace completion with failure

level: boundary area: Response terminal state

1.8.3 Asynchronous application composition

- The async recipe uses a shared supervised client and an application-bounded job count. Each job's worker opens and reads its own body inside one cancellation scope. A monitor-only guardian kills the worker when its application owner dies. The recipe adds no library body server or task runtime.
- The sink runs synchronously before the next demand. It owns retained state, framing and semantic progress. EOF can precede sink completion; an HTTP deadline cannot interrupt arbitrary sink work. Application shutdown cancels then kills its own worker after finite grace if necessary.
- Job await timeout preserves the job. One creating process consumes one completion result. Job copies are not repeatable futures or ownership transfers. One job finishing never stops the shared HTTP client. The preserved executable recipe is `examples/async`.

1.9 Budgets and effect timing

Independent capacities and clocks apply at named points.

Resource	Default	Admission or timing point
Connections	16 total; 4 per origin	Before resolution/Gun open; preparing reservations count

Resource	Default	Admission or timing point
H2 streams	100 per connection	Before submission; reduced by peer SETTINGS
Open bodies / queued requests	128 / 128	Before owner creation / pending queue insertion; finished handles and H1 checking slots count
Request body	1 MiB	Before admission; caller has already allocated bytes
Headers	16 KiB; 100 pairs	Request validation; informational/ final/trailer admission after parsing
Buffered response	128 KiB	Before appending each delivered chunk
Collected response	8 MiB	While send/body.collect/batch consume
Batch	10,000 inputs; 1–1,024 workers; 64 MiB	Before worker creation / shared charging before retaining collected chunks
Script data	16 MiB estimated	Before playback actor startup; not measured VM memory
Cassette read	Caller max_bytes	Read at most max_bytes + 1 raw bytes; parse checks encoded string length
Recording	16 MiB encoded; 10,000 exchanges	Before reservation/fragment enqueue; held redaction bodies charged
Connect	5 seconds	One absolute budget from DNS start through TCP/TLS
Pool wait	5 seconds	Admission to lease; new connection starter can use connect wait
Request	30 seconds or view replacement	Invocation entry through response consumption; Infinity explicit
Idle read	30 seconds or view replacement	While opener waits for head or reader waits on empty queue; events reset activity
Pooled idle / shutdown drain	60 seconds / 5 seconds	Ready unused connection / stop before forced local close
Recording waiter	One	Wait timeout/death clears only the waiter; finalization continues

- The connect timer includes DNS, TCP and TLS. Native connect/handshake limits receive its remainder. DNS worker lifetime ends under the connection budget; pending request expiry discards unused preparation. Idle read is active only while demand is outstanding, rather than while application code holds a chunk.
- Native TCP/TLS send_timeout and send_timeout_close use startup idle timeout. Request-owner expiry independently bounds fresh and reused buffered uploads. H1 expiry closes its exclusive connection; H2 stream cancellation preserves healthy siblings. No bound is claimed when the corresponding option is Infinity.

Boundary	Effect committed or observed	Failure and uncertainty
Validated invocation	No network action	Invalid/refused/capacity failure has NotSent
Resolution admitted	Address set authorized; no HTTP submission	Connecting failure remains NotSent
Gun request invoked	One asynchronous transport request issued	Every later live failure is MaybeSent; return proves no socket write or remote receipt
Final head admitted	Status and headers observed	Streaming caller owns head; collection failures retain redacted headers/status
HTTP EOF	Transport lease may release	Application consumption and capture finalization may remain incomplete
Capture acknowledgment	Writer fragment and close completed	No fsync or power-loss persistence established

Boundary	Effect committed or observed	Failure and uncertainty
Atomic link/rename	Completed cassette visible at destination	Abort cannot undo commit; a racing interruption can leave visible publication despite uncertain finish acknowledgment
Observation delivery	Best-effort notification	No HTTP/capture authority or durable history

- These are admission/storage and scheduling guarantees. User callback allocations, filesystem blocking, native buffers and VM scheduling remain outside them. Native timers do not create real-time execution guarantees.

1.10 Collection batching and failures

Preserve per-position outcomes and caller-owned retry decisions.

1.10.1 Collection and overflow

- `send` consumes the same body owner as streaming and always closes it. Buffered contains Response(BitArray), trailers, H1/H2/Offline and a truncated flag. Complete bodies concatenate accepted chunks; no UTF-8/JSON assumption is introduced.
- Fail overflow closes and returns LimitExceeded(ResponseBodyBytes, limit, observed), with final status and redacted response headers. Truncate closes and returns the first limit bytes, original status/headers, truncated true and empty trailers. It makes no claim to full response completion.
- `body.collect` explicitly supplies its own byte bound to an already opened body. Streaming next calls are not cumulatively limited by the collection limit; buffered-byte and time limits still apply.

BEHAVIOR

Collection overflow preserves the response head

Given the response head has arrived and collection uses Fail

When accepted response bytes exceed the collection limit

Then collection fails and closes the body

Then the failure retains status and redacted response headers

level: boundary area: Buffered collection

1.10.2 Bounded batch lifecycle

- A Batch validates input count and concurrency before scheduler startup. Workers own one request's open/read/close lifecycle. At most requested concurrency workers run. Caller death kills its workers; body ownership then releases their HTTP resources.
- Results preserve input order regardless of completion order. One request failure occupies only its position. Worker crash conservatively yields RequestFailed(UnknownTransport)/MaybeSent at that position and releases its worker slot.
- One shared atomic counter charges retained body bytes before collection keeps a chunk. Overflow fails the crossing response. Once crossed, positions not yet sent fail with BatchBytes/NotSent. Existing workers can settle independently. The counter is monotonic and does not refund failed-response bytes.
- The 64 MiB budget measures charged body bytes, not headers, input values, worker stacks or transient concatenation copies. The rejected crossing chunk has already been delivered to a worker. It is not a whole-batch VM allocation certificate.

1.10.3 Failure meaning and serialization

- Submission evidence is NotSent before the live Gun request call and MaybeSent afterward. Offline scripted failures retain recorded evidence while their telemetry mode is Offline. Closing is local cancellation and never remote rollback.

Closed Kind	Reasons grouped here	Caller action
InvalidInput	InvalidRequest	Correct input or settings
Refused	DestinationRejected; ViewDestinationRequired	Choose authorized destination
Unavailable	ClientClosed; AdmissionFull; PoolTimeout; RecordingClosed	May retry if evidence permits
Network	ResolutionFailed; ConnectionFailed; RequestFailed	Inspect typed transport cause and evidence
TimedOut	ConnectTimeout; DeadlineExceeded; IdleTimeout	Retry only under evidence/idempotency policy
TooLarge	Measured LimitExceeded	Change bounds or consumption policy
CancelledLocally	Closed; Cancelled	Reconcile application intent

Closed Kind	Reasons grouped here	Caller action
Misuse	ReadConflict; WrongOwner	Correct ownership/concurrency
Playback	PlaybackMismatch; PlaybackExhausted	Correct explicit fixture

- Only Unavailable, Network and TimedOut are retryable through `is_retryable`. NotSent permits another attempt; MaybeSent requires caller-declared idempotency. This helper does not schedule retries, interpret Retry-After or validate business idempotency.
- `error.Reason`, `error.TransportCause`, `error.LimitKind` and `error.RequestProblem` may gain variants in a minor release, so a caller matches them with a catch-all arm. `Kind` is closed. ADR-0015
- Known transport categories include name resolution, refused/reset/closed/draining connections, certificate/TLS rejection, protocol, dependency header limit, timeout and unexpected protocol. Unknown terms remain UnknownTransport without exposing raw dependency text or stack traces.
- `name` is a stable machine identifier; `describe` omits free-form request/config/fixture data. Failure never contains request headers, URL, query or body. Collection may retain response headers after redaction; serialization stores those retained headers. Explicit `with_headers` authors safe data and does not automatically apply a redaction policy.
- `to_json` and `decoder` round-trip typed diagnostic data and evidence. Required measured sizes stay distinct from unavailable parser measurements. StartError, request Failure, cassette I/O, RecordError and FinishError remain separate boundaries.

1.11 Playback cassettes and redaction

Offline sequence ownership and one current data format.

1.11.1 Playback matching and cursor

- `testing.playback` validates complete configuration, whole-byte request/chunk bodies, final statuses 200–599 and estimated script size before starting. It creates a Playback session with one global admission-ordered cursor. Concurrency does not define a reusable first-match lookup.
- Exact match keys apply configured redaction to both expected and actual requests. Host and header names become lowercase, empty path becomes slash, ignored headers disappear and remaining headers sort by name while duplicate-name order remains. Method, URI fields and bytes otherwise remain exact.
- An explicit custom matcher receives these normalized forms. It owns only equality; common validation, lifetime and response admission still run. A mismatch reports the zero-based position and consumes nothing. Exhaustion fails without network fallback. Custom callbacks are trusted code and can block the pool.
- A loaded `testing.Script` (Script) is immutable input. Each `testing.playback` call starts a separate client at its first exchange, so one script serves independent runs. Client views share their client's cursor; `with_correlation` changes observation metadata and selects no sequence. Concurrent requests on one client consume exchanges in admission order.
- Offline requests skip destination resolution/address authorization and open no network socket. Required-view selection still applies. Bodies pass the same post-delivery header/byte admission and collection logic with negotiated protocol Offline.

BEHAVIOR

Playback mismatch preserves the next exchange

Given an offline session has an unused next exchange

When the request does not match it

Then the request fails with the current position

Then the next matching request consumes that same exchange

level: boundary area: Playback matching

1.11.2 Cassette schema and bounded file input

- A Cassette has marker `http_gun: 2` and an exchanges array. Each exchange has method, URL, ordered header pairs and body; its reply is reject(failure) or response(status, headers, chunks, ending). Ending is finished(trailers), aborted(failure) or abandoned.
- Whole bytes encode as text when valid UTF-8, otherwise base64. Invalid required fields, methods/URLs, header-pair shapes, base64 or final status produce Corrupt. Another integer format marker produces UnsupportedVersion. There is one current decoder and no compatibility fallback. ADR-0005
- Required fields are decoded; unknown extra JSON fields are not a closed-object rejection contract. If both text and base64 are present, the text branch has precedence. Parse does not perform the playback script-data budget check until playback startup.

- `parse(text, max_bytes)` checks encoded string size before JSON decoding. `load(path, max_bytes)` uses raw reads of at most max_bytes plus one without read-ahead. A negative file limit clamps to zero. Empty/non-UTF-8/malformed data is corrupt; missing and I/O failures remain typed.

1.11.3 Secret ownership and whole-body rewriting

- Redaction always removes authorization, proxy-authorization, cookie, set-cookie, x-api-key, api-key and x-goog-api-key. Added header names are case-insensitive and affect stored request/response headers, trailers and collected failure headers.
- Query redaction matches exact percent-decoded parameter names, treating plus as space. Listed names keep their original encoded spelling with value REDACTED. Parameter order and all unselected query bytes remain exact.
- A body hook rewrites each request body and each response body as a whole. Recording holds response chunks within its capture budget then stores one rewritten chunk, so split secrets are visible to that hook. A later hook replaces the earlier one. The live request/response delivery remains unchanged.
- Matching applies the same hook to expected and incoming request values. A recorder can redact a request before storage and matching can redact it again; callback authors therefore supply stable repeatable rewriting. Callbacks own their execution time, allocations and retained captures. There is no general secret detector or callback sandbox.
- Unlisted headers, query parameters and body data remain in fixtures. Safe logging of failures and observations does not imply that user-supplied fixtures, callback state, native VM crash data or arbitrary application logs contain no secrets.

1.12 Recording and filesystem publication

Capture completion is separate from HTTP completion.

1.12.1 Reservation order and acknowledged writer

- `cassette.record` validates configuration and capture options, creates one exclusive sibling staging directory and applies mode 0700 before request data is stored. It then starts the recorder and live pool. Failed pool startup discards the recorder's staging area.
- Each Capture reservation receives a zero-based position before live execution. Concurrent completion writes separate exchange spool files; final assembly follows reservation order. Capacity/policy/connect/open failures can be recorded as reject outcomes. Refusals preceding capture reservation are not stored exchanges.
- The recorder has one active writer and an admitted job queue. Request fragment reservation and each head/chunk/terminal fragment are budgeted before enqueue. Body owners wait for capture acknowledgment before delivering successful data or demanding more; capture limits also bound held whole-body redaction input.
- Acknowledgment follows completed normal write and file close. There is no delayed-write buffer or fsync guarantee. Capture failure detaches recording from HTTP; live outcomes continue independently. A recorder closed by accepted finish refuses new requests with RecordingClosed; a broken capture may let live requests continue uncaptured.
- A body with pending capture acknowledgment after HTTP EOF can retain its timer until capture settles. Deadline may separately abandon capture without retrospectively changing completed HTTP. Capture is not an eager HTTP reader and cannot drain an unread response to make finish succeed.

1.12.2 Finish waiter and publication commit

- Nonpositive finish waits request immediate completion. Active unfinished capture returns Busy without closing new reservations. Positive waits close reservations, wait for all streams/jobs and initiate final assembly. Only one waiter is accepted; concurrent contention returns Busy.
- WaitTimeout or waiting-process death removes only the waiter. Closing/Sealing continues; a later finish sees the final result. The recorder's creator death interrupts unfinished capture. Normal live-client stop does not itself discard completed capture; abnormal client loss interrupts it.
- Sealing builds a complete JSON file beside the destination from bounded spool reads. RefuseExisting publishes by hard link; ReplaceExisting publishes by rename. No check-then-overwrite fallback exists. AlreadyExists and other I/O causes remain explicit. ADR-0005
- Publication is atomic on a filesystem supporting the selected link/rename operations. Cross-filesystem publication and power-loss survival are not promised. Published data is complete; interrupted staging data is not a replayable cassette.
- Abort is local to capture and leaves the live client usable. It cannot undo an already committed filesystem operation. The actor only observes publication through writer acknowledgment, so an abort/owner-loss race after atomic commit can leave a visible file while finish reports interruption. No stronger transaction barrier is inferred.

BEHAVIOR

Finish wait expiry preserves finalization

Given accepted finalization is waiting for existing exchanges

When the finish wait expires

Then the caller receives WaitTimeout

Then finalization continues without accepting new reservations

Then a later finish can observe its final outcome

level: boundary area: Recording finalization

1.12.3 Failure cleanup and operational limits

- Write failure, capture overflow, abandoned reservation, failed publication, abort, owner loss and recorder crash trigger staging cleanup. A janitor monitors recorder death. Cleanup kills the active writer and waits up to 200 milliseconds before allowing a separate cleaner to wait for its exit.
- The actual removal primitive deletes only direct files or empty child directories, then the staging directory. It never recursively traverses unknown nested data. Removal errors are ignored. Unremovable entries, a blocked writer and VM termination can leave data behind. The cleanup promise strength is unresolved in Pending updates. ADR-0010
- file_streams supplies explicit Raw/Exclusive writes and bounded raw reads. simplifile supplies permission, hard-link, rename and file deletion. Scope helpers close handles on exceptions before re-raising. When both normal operation and close fail, the primary operation failure takes precedence.
- Runtime leftovers in a **Staging directory**, named with the destination's `.http-gun-` staging suffix are safe to remove after the writer/recorder no longer owns them. Applications decide artifact retention and privacy. No directory sync or remote object-store publication port exists.

1.13 Observation and composition ports

Identity and milestones carry facts without granting authority.

1.13.1 Milestones and delivery selection

- Lifecycle event path is `http_gun/lifecycle`. Its typed metadata carries **Request id**, optional caller correlation, optional client label, Live/Recorded/Offline mode and a milestone. Timing is monotonic VM-local milliseconds.

Milestone	Authority	Observed fact
AdmissionEntered	Pool	Validated invocation received before admission/capture reservation
AdmissionWaiting	Pool	Invocation entered finite waiting/preparation work
AdmissionGranted	Pool	Eligible lease selected or offline exchange matched
GunCallReturned	Body owner	Asynchronous Gun API returned; no transport write/receipt established
ResponseHeaders(status)	Body owner	Final head passed admission
HttpTerminated(outcome)	Body owner or rejecting pool	Complete, LocallyCancelled, DeadlineExpired or Failed

- No lifecycle event contains URL, query, header, body, credentials or raw dependency terms. There are no per-chunk events by default. Labels and correlation are application-supplied metadata and must themselves be safe.
- Default Emit calls `sinal.emit` and follows application routing. Without a route, user handlers run synchronously inside pool/body actors and can delay requests and timers. A configured direct forwarder isolates delivery; full/unavailable forwarding is best effort and its result is discarded. Silent emits nothing. ADR-0006
- HTTP Gun creates no collector, retained timeline, forwarding mailbox or export queue. Applications supervise and bound forwarders and history. Observation arrival order across pool/body processes is not a request-order guarantee. Abrupt actor/VM death can omit terminal notification.
- Events are unsuitable for effect correctness or retry permission. Complete means HTTP EOF rather than application consumption or successful publication. Offline never emits a live Gun submission milestone. Recording persistence is a separate authority.

1.13.2 Caller and extension obligations

- An application routes the `http_gun` event prefix to a forwarder at startup so handlers never run inside the pool. A node-wide handler sees the events of every client, including those libraries own, and filters on the client label.
- A library receiving a view reads `http_gun.correlation` and copies it into its own event vocabulary. It preserves caller settings. A library owning a private client labels it with the library name; silence is an explicit alternative. Application startup owns route installation and removal.
- Resolver, body redactor, playback matcher and scoped response callback are the variable-behavior ports. Common client admission, failure evidence and actor cleanup remain local to HTTP Gun. Callback authors own finite captured state and resources they spawn; custom matcher/redactor execution is not isolated by a timer.
- Provider adapters own authentication, request encoding, expected status/media type, cumulative semantic response bounds, SSE/provider framing and outcome reduction. MCP clients own MCP correlation/protocol semantics. OIDC clients own URL/trust restrictions and identity validation. No shared retry/runtime/error/durability abstraction is introduced.
- Body transfer has no public operation. An async consumer opens inside the process that will read. Introducing transfer would require revocation, in-flight read, death and cancellation semantics; it is not inferred from body copying.

1.14 Foreign boundary and dependency upgrades

Translate native operations without moving policy out of Gleam.

Bridge	Allowed responsibility	Upgrade-sensitive contract
Gun bindings	Start/open/request/update_flow/cancel/close, supported info lookup and terminal event forwarding	Asynchronous request submission; protocol choice; no replay; H1 state_name inspection; terminal cause sent to the original owner before Gun exits
Event conversion	Up/down/head/data/trailers/inform/upgrade/settings conversion to typed events	Structured error shapes; H2 pseudo-header counting; initial capacity changes
Runtime primitives	Monotonic time, atomic batch counter, exception-safe scopes and address/DNS conversion and OTP PEM/DER identity admission	VM-local clocks; whole-byte/IP conversion; narrow trusted boundaries
Filesystem primitives	Unique directory candidate, empty-directory/direct-entry cleanup	Exclusive creation remains in Gleam; nonrecursive best-effort removal

- No handwritten Erlang pool, body server, recorder server, DNS admission server or protocol parser exists. Native exception classification returns bounded recognized causes and UnknownTransport. Internal module visibility is documentation organization rather than an import-security boundary; opaque capability construction and actors enforce authority.
- Gun range is 2.6.0 inclusive to 2.7.0 exclusive. Cowlib range is 2.21.0 inclusive to 2.22.0 exclusive. Patch ranges permit security updates while minor changes require deliberate audit. The committed manifest selects the tested minimum; latest-patch isolated resolution exercises the other end. ADR-0007
- Undocumented assumptions are H1 `gun:info` state_name connected; reason terms inside gun_error/gun_down; Gun terminal event callback before process exit; source-reviewed request return timing; H2 max_headers including status; and test-only cow_http2/cow_hpack calls. Unknown state rejects reuse; unknown reasons lose precision rather than becoming safe-retry evidence.
- Widening a range requires inspecting each assumption, updating decoding only under an accepted contract, running minimum/latest full gates and qualifying public consumers. A changed test helper API can break H2 evidence and must not be mistaken for a production protocol change.

1.14.1 Native assumption inventory

Assumption	Source	Behavior if it changes
`gun:info/1` returns `state_name: connected` and protocol `http` for a reusable H1 connection	`src/http_gun_ffi.erl:reusable/1`	A missing or unknown state refuses reuse; the replacement connection stays before submission
Structured `gun_error` and `gun_down` reasons	`cause/1`, `decode/1`	UnknownTransport loses detail; an unknown reason never yields NotSent
Gun calls its event handler with the terminal reason before process exit	`src/http_gun_event_h.erl:terminate/2`; Gun `terminate/3`	The original owner and typed reason are retained when the owner installs its monitor after Gun exits

Assumption	Source	Behavior if it changes
The request call returns asynchronously before socket processing	`request/5`; reviewed Gun request implementation	MaybeSent begins at invocation; no hidden retry
H2 header count includes `:status`	`open/10`; Gun and Cowlib source	Ordinary-header admission stays exact after parsing
Settings-change notification and peer stream capacity	`decode/1`; `internal/pool.gleam`	No H2 stream opens before initial SETTINGS; a reduction blocks new admission
Test-only `cow_http2` and `cow_hpack` operations	`test/http_gun_h2_server.erl`	The H2 test server is requalified before any H2 behavior claim
Gun wraps a post-handshake TLS alert as `{closed, {tls_alert, {Alert, Detail}}}` for stream owners	Gun `gun_http:close_reason/1`, `gun_http2:close_reason/1`	The nested recognized cause is decoded without exposing `Detail`; MaybeSent is preserved after submission

- A range widening runs the full gate on the committed minimum and on the newest admitted patches through the isolated environment. CI selects latest patches with `HTTP_GUN_DEPENDENCIES=latest`. The selected root Gun and Cowlib versions are imposed on the disposable ordinary, async, mTLS and stdlib consumer copies; the gate fails when a resolved consumer differs and retains the manifests under `build/evidence/manifests`.
- Public consumers and explicitly selected downstream checkouts are requalified when a public contract changes. Qualification uses no provider credential and patches no dependency.
- The runtime matrix qualifies the Gleam OTP and Erlang process APIs that own native lifetime. A separate consumer build and the complete fast suite run at the advertised stdlib lower bound and at the selected current version. The disposable lower-bound package resolves a Gleeunit version compatible with its stdlib inside the existing dev-dependency range and retains that manifest. Authored Erlang warning rejection stays separate from upstream diagnostics, so an older stdlib can emit OTP deprecation diagnostics while package-owned source still passes `erlc -Werror`.
- `file_streams` preserves explicit Raw reads without read-ahead, Exclusive writes and close error reporting. `simplifile` preserves hard-link and rename publication and permission semantics. A check-before-create helper never substitutes for exclusivity. A durability promise beyond atomic visibility requires its own supported-platform contract, because file sync alone does not establish directory-entry persistence.

1.15 Validation and retained capability scope

Observable tests establish boundaries; preserved intent remains visible.

1.15.1 Evidence and tests by boundary

Boundary	Executable evidence	What it establishes
Ordinary and API adoption	`examples/ordinary`, README tests, type rejection in dev/consumers	Common calls, advanced views, caller errors, opaque capability use through public imports
HTTP/H1/TLS	test request/stream/reuse/trust/destination wire suites; local Erlang servers; separate mTLS consumer	Binary/status/header/trailer fidelity, reuse, deadlines, policy, write cleanup
H2	Controlled H2 server plus dev/nghttpd	Actual negotiation/shared socket, peer capacity, cancellation and GOAWAY boundaries
Lifecycle and async	Timeout/pool/adoption suites; examples/async	Read conflicts, owner death, supervision, local wait, blocked sink and cancellation
Cassette and capture	Cassette/recording/fault suites	Strict order, byte codec, terminal failures, redaction, capture independence and atomic publication
Finite resources	Batch/recording/load benchmarks and dev boundary checks	Admitted counters, captured budgets and observed scaling under stated environment
Dependency boundary	FFI warning build, min/latest CI, stdlib boundary checks	Supported source compatibility and native helper assumptions

- Gun/Cowlib source and protocol behavior are authoritative for transport mechanics. Dream, ReqCassette, Gleam HTTPc, Mint and Finch supply scoped scenarios and comparison methods. They are not complete parity oracles. The oracle document

pins each reference and states what is compared; attribution, hashes and raw receipts remain with executable evidence. ADR-0001

- Source inspection, inspired tests, faithful ports and differential execution are distinct evidence classes. Real H2 sharing cannot be proven by two H1 sockets or script playback. Performance samples establish only their workload/runtime measurements.
- Normal gates use local HTTP/TLS/H2 servers, temporary files and no provider credentials. Loopback access is enabled explicitly by each local test client.

1.15.2 Validation profiles and gates

Profile	Command	Content
Fast	`./dev/env sh dev/gate fast`	Formatting, Gleam check/build/tests, native warnings, ownership and dependency boundaries
Full	`./dev/env sh dev/gate full`	Fast plus stdlib edge versions, ordinary/async/mTLS consumers, real nghttpd HTTP/2, shared batch budgets, recording and load observations
Matrix	`sh dev/matrix`	Each selected runtime shell qualifies in isolation
Soak	`sh dev/gate soak`	Ten-minute local nghttpd soak in the extended workflow

- `dev/env` enters the pinned Nix shell. A Validation profile selects the checks and evidence of one gate run. `dev/gate` builds an isolated validation copy unless it already runs inside that gate; normal gates consume no sibling working copy. The outer invocation checks the original Git tree with `nix flake check --print-build-logs` and runs gate controls before creating the copy. Ignored local credential files never enter Nix source snapshots. ADR-0014
- The Nix checks enforce canonical Gleam, Nix, Elixir, Markdown, YAML and Python formatting and authored-tooling lint (actionlint, ShellCheck, Ruff restricted to E9/F rules on authored `dev/**/*.py`). Formatting excludes generated build, render and PDF output, retained `docs/evidence` data and frozen `dev/dependencies` bytes. Compiler-negative fixtures live only in temporary consumer copies.
- `dev/check_native.py` compiles authored Erlang, including both consumers' FFI and copied local servers, with `erlc -Werror`, dependency include paths and generated dependency BEAM paths. A valid native module and an intended unused-variable rejection test qualify this check separately from Gleam's warning flag.
- `dev/check_results.py` rejects missing, empty, failed, skipped or ambiguous test summaries and requires all eleven distinct load workloads with their H1/H2 connection and large-byte assertions. Controls reject malformed JSON, duplicate or missing workloads and stale consumer dependency resolution. A timeout control proves isolated gate descendants stop before workspace cleanup.
- Each gate run replaces `build/evidence`, so a failed run cannot export an earlier success receipt. `build/evidence/logs` keeps commands and raw per-check logs, `build/evidence/manifests` keeps the resolved root and consumer manifests, and `summary.json` records the profile, selected dependencies, elapsed time, outcome and exit code, failed runs included.
- CI runs six full-gate lanes (OTP 27, 28 and the default OTP 29 shell, each with the committed minimum and the newest admitted Gun/Cowlib patch) and a separate design job. The final CI job accepts only successful matrix and design results. Every push, pull request and manual dispatch runs it with pinned actions, read-only repository permission, superseded-run cancellation and bounded job times; failure evidence is retained for 14 days.
- A weekly or manual extended workflow runs the soak, native ARM64 full qualification and the pinned public comparison harness, retains results for 30 days and reports its own final status. The batch scaling guard is `5000 requests < 15 × 500-request median + 50 ms`; measurements add no absolute latency promise.

1.15.3 Public consumers and local fixtures

- `examples/ordinary` compiles as its own package. One runnable module per topic is the usage reference: configuration and validation, client views with streamed bodies, limits and batches, destination policies and tenants, scripted playback with redaction, and the supervision, failure, observation and record/replay flow. The full gate builds and runs them, and README snippets compile in the fast gate, so a changed name or behavior fails a gate. Its cancellation checks exercise retained application and operation tokens before submission, while queued and during a live response; duplicate addition, replacement, source-token independence and borrowed-client reuse are separate assertions.
- `examples/mtls` runs ephemeral RSA and EC identities against a real local TLS peer. It qualifies H1/H2 authentication, rejection, recovery, per-client connection isolation, snapshot reconnection, rotation, unchanged server and destination checks and credential-free capture. This is local native evidence, not provider certification.
- `examples/async` owns the bounded application job recipe: worker lifetime, synchronous sinks and cancellation. Its worker opens and reads the body through `with_response` in one process, runs the sink before the next read and holds no forwarding loop or body mailbox. Early Stop returns the observed prefix, a guardian kills the worker when its creator dies, and shutdown cancels, waits a finite grace and kills only that worker.

- `dev/check_downstream.py` runs explicitly selected, isolated LLM Wire qualification and records source identity. It writes to a fresh receipt directory, excludes credential names before reading or copying source and stays outside the ordinary gate. Archived consumers prove only their recorded revisions; a current consumer requires selecting current source and checking that the originals remain unchanged. No normal gate modifies a sibling checkout.
- Local Erlang HTTP, H2 and TLS servers use the test certificates; `dev/ip-fixture` generates the IP-SAN case. `dev/nghttpd.py` drives the development nghttpd package for real peer-capacity and trailer checks and owns no provider semantics. A native connection-refusal regression waits for Gun to exit before installing a second monitor, so the monitor reports `noproc` while the preceding terminal event retains ConnectionRefused.
- `build/evidence` holds current runtime output. `docs/evidence` preserves the raw source, hash and measurement data that `docs/ORACLE.md` and `docs/BENCHMARKS.md` cite; those paths carry no design contract. `NOTICE`, dependency licenses and fixture attribution remain authoritative for source reuse.
- `dev/convert_cassette.py` is an explicit offline schema-1-to-2 conversion tool. Runtime playback has one decoder and never migrates or rerecords.

1.15.4 Comparison and benchmark method

- The comparison harness in `examples/comparison` contrasts the HTTP Gun and Dream public APIs over local HTTP/1.1 servers. Its adapter retains status, error and body differences and gives the workload driver one call shape. Its contract suite covers binary responses, status handling, duplicate headers, trailers, local TLS, early closure, owner exit, explicit callback cancellation and disk record/playback; these are behavior observations separate from timing.
- Each timing case uses three fresh BEAM VMs, warms the transport and alternates client order. Elapsed time excludes compilation and VM startup. The sampler reads whole-VM memory, mailboxes, processes and ports about every 10 ms and can miss short peaks; connection counts are distinct server-side identities, not peak simultaneous sockets.
- The harness runs the current HTTP Gun source, so a rerun produces new evidence rather than reproducing retained source hashes. Three local H1 trials establish neither a performance ranking nor a latency or allocation guarantee. ADR-0013
- The oracle document holds the pinned references and the comparison classification, and the benchmark document holds the run commands and the dated retained measurements. The two link each other; this design states the method rules and never a result.

1.15.5 Retained extensions and explicit exclusions

- Streamed uploads, redirect following, decompression, proxies, cookie/cache adapters and generic SSE framing remain intended extensions with their contracts in Pending updates. The present facade returns buffered uploads, redirect statuses and encoded response bytes directly. ADR-0009
- WebSocket, HTTP/3, server-framework, automatic application retry, transfer recovery and agent continuation remain outside package ownership. Gun's broader capability does not expand this facade implicitly.
- A broader extension must preserve process/body ownership, address checks on new routes, one-attempt evidence, finite admission and independently reported capture. Cross-package consumers prove the public extension from their own package before facade acceptance.

Glossary

Every term the layer declares, with the context that owns it.

Batch

owned by root

A set of requests that run with bounded concurrency under one shared retained-byte budget. Results keep input order and each failure occupies only its own position.

Body capability

owned by root

A process-owned reference to one response cursor and terminal outcome. Copies refer to the same read and close authority.

Cancellation token

owned by root

A process whose death marks every copy of the token cancelled. A client view retains one or more tokens, and any retained token cancels requests through that view.

Capture reservation

owned by root

An exchange position reserved before live execution for a recording. Positions establish fixture order independently of completion order.

Cassette

owned by root

A versioned JSON value containing ordered HTTP request and reply observations. It is data rather than a live network or playback capability.

Checked answer

owned by root

The complete address set admitted for one new connection. Its authority lasts for that connection rather than for the hostname indefinitely.

Client identity

owned by root

An immutable snapshot of a PEM certificate chain and one unencrypted private key, bound to one client's TLS connections. Views cannot replace it.

Client view

owned by root

An immutable HTTP client handle carrying per-call settings over the same pool. Copies share pool lifetime while their view settings remain independent.

Destination policy

owned by root

The rule set that admits a request's host, port, address classes and plaintext use. A client holds one policy and each view appends a narrowing policy.

Exchange

owned by root

One request attempt with its observed response or failure. It has local identity and supplies no remote idempotency guarantee.

Flow credit

owned by root

Permission for Gun to deliver a response data event. It counts messages rather than bytes or parser allocations.

HTTP termination

owned by root

The response owner's terminal HTTP observation: completion, local cancellation, deadline expiry or failure. It does not certify application consumption or capture publication.

Open body slot

owned by root

A retained-body admission slot occupied until the body closes or its process exits. HTTP termination can release a stream lease without releasing this slot.

Origin

owned by root

The canonical tuple of lowercase unbracketed host, effective port and TLS choice. It identifies a connection lane within one client's immutable connection policy.

Origin lane

owned by root

A first-in-first-out queue of pending requests that share one Origin. A blocked lane head blocks only its own lane.

Playback session

owned by root

An offline client's ordered consumption cursor over a validated script. A mismatch does not advance that cursor.

Publication

owned by root

The atomic filesystem operation making a complete cassette visible at its destination. Visibility does not establish persistence through power loss.

Read wait

owned by root

One pending demand for the next body event. Its optional local expiry ends that wait without ending the exchange.

Recording session

owned by root

An owner of bounded captured exchanges and the authority to finalize their fixture. Its result is independent of the live HTTP result.

Redaction

owned by root

The selected removal or rewriting of data retained in fixtures and failure headers. It preserves the live request and does not detect all secrets.

Request id

owned by root

An opaque random UUID v4 for one observed invocation, probabilistically unique across nodes and restarts. It is distinct from the caller's correlation and from a durable remote request identifier.

Script

owned by root

An immutable ordered list of expected requests and their replies. Each playback client consumes its own copy from the first exchange.

Shutdown grace

owned by root

The interval in which `stop` lets active leases finish before it closes the remaining bodies and connections locally. It bounds the planned drain only.

Staging directory

owned by root

The directory beside a cassette destination, named with the `.http-gun-` suffix, that holds unredacted capture data until publication or cleanup.

Stream lease

owned by root

One exchange's claim on transport capacity. It occupies one exclusive HTTP/1.1 connection or one HTTP/2 stream.

Submission evidence

owned by root

The distinction between an attempt known to precede the transport request call and one that may have reached the peer. Neither alternative proves remote execution.

Validation profile

owned by root

A named gate configuration (fast, full, matrix or soak) that selects which checks run and which evidence the run retains.

View destination

owned by root

A per-call policy that selects hosts or refuses an address class admitted by the client. A plaintext-only restriction does not select a destination.